Building data course — Week 3 — fault detection in SQL and pandas

sparql-playground / lessons

Generated for print and offline reading

Week 3 — fault detection in SQL and pandas

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. The scripts are in the repo next to each lesson. Keep notes under the ignored `student-work/` folder.

Contents

- Day 11 — Indexes and EXPLAIN QUERY PLAN: why some lookups are instant and others scan everything
- Day 12 — Window functions: look at neighbours and rolling windows (LAG, LEAD, rolling mean, fan starts)
- Day 13 — Streaks (gaps and islands): how long has this been true?
- Day 14 — Write a fault rule (FC1, duct static too low) twice, and check it against Open-FDD
- Day 15 — Data quality before you trust a rule, and the Unit 1 challenge

---

Day 11 — Indexes and EXPLAIN QUERY PLAN: why some lookups are instant and others scan everything

Goal: read a database's query plan, see the difference between a seek and a scan, build an index and measure what it buys, then do the same three ways in pandas (mask, `.loc` on a sorted index, binary search).

Read

Every query has two parts: *what* you want and *how* the engine finds it. SQL only states the *what*. The *how* is the **query plan**, and SQLite shows it if you put `EXPLAIN QUERY PLAN` in front of the statement. Two words matter:

- **SEARCH**: the engine used an index (a B-tree) to jump to the matching rows. About O(log n) to find the first match, plus O(k) to read the k rows that match.
- **SCAN**: the engine read every row and tested each one. O(n), every time you ask.

FDD means asking the same kinds of questions over and over: "everything in March", "everything where static is high". Whether those are seeks or scans decides if a rule over a few years of data takes milliseconds or minutes.

Today's two questions, both on AHU_1's 35,536 rows:

1. **A time range:** one week of rows (1 to 8 May). `timestamp_utc` is the real table's `PRIMARY KEY`, so SQLite already keeps a B-tree sorted on it. To see the difference an index makes, the SQL script works on a `TEMP` copy of the table (so the lessons database is not changed). A copy has no

primary key and no index, so the plan says `SCAN copy`: test every row. Then the script runs `CREATE INDEX` on the copy's `timestamp_utc`, and the plan flips to `SEARCH copy USING INDEX copy_time (timestamp_utc>? AND timestamp_utc<?)`. (The timestamps are text in one fixed format, so text order *is* time order and a range works on text.)

2. **A rare value:** rows where duct static is above 1.7 inH2O (32 rows). Nobody indexed `da_p_inwc`, so this is a scan: all 35,536 rows read to return 32. Try it 2 builds an index for it.

pandas has no query planner. *You* choose the access path, and there are three:

Question	SQL (`day11_sql.py`)	pandas (`day11_pandas.py`)
2. look at every row	`SCAN` (no index)	a True/False mask `(df["timestamp_utc"] >= start) & ...`, O(n) every time
3. build a sorted structure once	`CREATE INDEX`	`df.set_index("timestamp_utc").sort_index()`, O(n log n) once

Question	SQL (`day11_sql.py`)	pandas (`day11_pandas.py`)
3. jump to the range	`SEARCH ... USING INDEX`	`.loc[start:last_row]` on the sorted index, O(log n + k)
do the jump by hand	(the B-tree does it)	`np.searchsorted(sorted_array, a)`, O(log n) per bound

`.loc` on a sorted index includes its end label, so the pandas script names the week's last row (`2026-05-07 23:55:00+00:00`) rather than the next midnight.

`np.searchsorted` is plain binary search: halve the sorted array until you find the spot. A B-tree is the same idea made friendly to disks (many keys per node, so fewer steps), and it stays sorted as rows are added, which an array does not.

Big-O intuition. A mask touches every row, so it costs about c·n per query. A sorted structure costs a **one-time build** of about n·log n, then about log n + k per query. If the build costs B, one scan costs S and one lookup costs L, the index pays for itself after about B / (S − L) queries. Both scripts print B, S and L, so you can work it out: it is small here (a

handful of queries) because the build is cheap. At 100 million rows the build costs more in absolute time, but the scan grows 1000× while the lookup barely moves, so the index wins by more and more. If you will only ask once, don't build an index: a scan is cheaper than build plus lookup.

An index is not free, and not always useful. It takes space, it must be updated on every insert (slowing writes), and it only helps queries that are *selective*. Asking for 40% of the table through an index means jumping around for 14,000 rows, and reading the table straight through is about as fast. Try it below.

Run

```
uv run python lessons/course.py run 11
```

That runs `day11_pandas.py`, then `day11_sql.py`. The row counts and the average must match; the **timings differ** from run to run and machine to machine (the tests compare only the answers). Both print:

```
Rows in the week: 2016
Rows in the week (index lookup): 2016
Average duct static in the week (inH2O):
0.5701
Rows with duct static above 1.7 inH2O: 32
```

then their own timings in milliseconds (scan, index lookup, building the index once), and the SQL script adds its two plans:

```
Plan before the index: SCAN copy
Plan after the index:  SEARCH copy USING
INDEX copy_time (timestamp_utc>? AND
timestamp_utc<?)
```

The week is 2,016 rows (7 days × 288 five-minute rows) both ways. The lookup is faster than the scan, and building the index costs about as much as one or a few scans.

Try it

1. In the SQL script, add `conn.execute("CREATE INDEX temp.copy_da_p ON copy (da_p_inwc)")` and print the plan of step 4's query before and after. What changes?
2. Now raise the threshold to `1.2` inH2O (about 14,000 rows). Does the plan change? Is the indexed query still faster than a scan?
3. Why can't `set_index("timestamp_utc")` speed up "`da_p_inwc > 1.7`" in pandas?

Answers

1. Before: `SCAN copy`. After: `SEARCH copy USING COVERING INDEX copy_da_p (da_p_inwc>?)`

("covering" because the index holds every column the query needs), and the 32 rows are found without reading the other 35,504.
2. 14,237 rows match. The plan still says `SEARCH ... USING INDEX`, but the time is about the same as a scan. An index only wins when it lets the engine skip most of the table. When 40% of rows match, the cost is dominated by reading and returning those rows. Selectivity decides, not the plan wording.
3. An index on the timestamp is sorted by timestamp, so it only answers questions about timestamps. Another filter needs another sort order: `df.sort_values("da_p_inwc")` (or `np.argsort` of that column, the row numbers in value order) would be exactly what a second database index is.

Why it matters

Real FDD queries are almost always "this equipment, this time range, this point". The database must jump straight there, or a rule scales with the *whole history* instead of the slice it needs. Production historians use a composite index on (series, time) for exactly this reason. Day 14's overlap check is a place

to try one. Day 12 uses windows to look at neighbouring rows, and Day 13 turns that into streaks.

Day 12 — Window functions: look at neighbours and rolling windows (LAG, LEAD, rolling mean, fan starts)

Goal: compare each row with the rows around it, in SQL window functions and in pandas `shift` and `rolling`: previous and next readings, a 1-hour rolling mean, and the moments the supply fan starts and stops.

Read

An ordinary SQL query treats each row alone. FDD rules constantly need a row's **neighbours**: "did the status just change?", "what was the reading five minutes ago?", "what is the average over the last hour?". A **window function** computes a value for each row from a *window* of related rows, without collapsing them the way `GROUP BY` does. The output has as many rows as the input.

```
LAG(duct_static_pa)  OVER (PARTITION BY
         equipment_id ORDER BY
         timestamp_utc)   -- the previous
         row's value
LEAD(duct_static_pa) OVER (PARTITION BY
         equipment_id ORDER BY
         timestamp_utc)   -- the next
         row's value
AVG(duct_static_pa)  OVER (PARTITION BY
         equipment_id ORDER BY
         timestamp_utc
                        ROWS BETWEEN 11
         PRECEDING AND CURRENT ROW)
         -- this row and the 11 before it
```

`PARTITION BY equipment_id` restarts the window for each AHU, so AHU_2's first row never sees AHU_1's last row. `ORDER BY timestamp_utc` is what makes "previous" mean *earlier*. `ROWS BETWEEN 11 PRECEDING AND CURRENT ROW` is a 12-row **frame**: 12 five-minute rows = one hour. The first rows of each AHU have fewer than 11 rows behind them. SQL just averages what exists. pandas needs `min_periods=1` to do the same (the default would give NaN until it has all 12).

Idea	day12_sql.py	day12_pandas.py
previous / next row	`LAG(x) OVER (ORDER BY timestamp_utc)` / `LEAD(x)` ...	`df[x].shift(1)` / `.shift(-1)`

Idea	`day12_sql.py`	`day12_pandas.py`
15 minutes ahead	`LEAD(x, 3) OVER (...)`	`.shift(-3)`
change since last row	`x - LAG(x) OVER (...)`	`x - x.shift(1)`
1-hour rolling mean	`AVG(x) OVER (... ROWS BETWEEN 11 PRECEDING AND CURRENT ROW)`	`.rolling(12, min_periods=1).mean()`
fan start	`fan = 1 AND LAG(fan) OVER (...) = 0`	`(fan == 1) & (fan_prev == 0)`
first row has no previous	`LAG` is `NULL`, so the comparison is not true	`shift` gives `NaN`; the comparison is `False`

The scripts look at AHU_1 only, so they need no `PARTITION BY`. With both AHUs stacked in one table you would add it (and `df.groupby("equipment_id")[x].shift(1)` in pandas). Both are one pass over rows in time order, so O(n) (plus the sort). The rolling mean can look like it costs 12 operations per row, but both engines slide the window, adding the new row and dropping the oldest, so it stays O(n).

Rows, not time. `ROWS BETWEEN 11 PRECEDING` and `rolling(12)` count rows. They mean "one hour" only because the data has exactly one row every five minutes. Day 15 checks that, and Day 18 handles data that has gaps.

Units. The CSV logs duct static in inches of water, so the scripts convert to pascals first (`da_p_inwc * 249.08891`), and Day 12's numbers are in Pa. SI is what the fault rules use from here on.

Run

```
uv run python lessons/course.py run 12
```

That runs `day12_pandas.py`, then `day12_sql.py`. Both print:

```
AHU_1 fan cycling (duct static in Pa)
  fan starts: 123
  fan stops:  122
  highest 1-hour mean static (Pa): 427.7
  first fan start: 2026-03-16
09:00:00+00:00
  average static 15 min after a start
(Pa): 294.68
First 3 fan events (start=1 or stop=1):
            timestamp_utc  fan_start
fan_stop
2026-03-16 09:00:00+00:00          1
0
2026-03-16 23:05:00+00:00          0
1
2026-03-17 08:35:00+00:00          1
0
```

(Times are UTC.) The fan starts about once a day and runs about 14 hours on a normal day. Fifteen minutes after a start the static averages about 295 Pa, already close to where it settles: the fan comes up to pressure quickly. The 1-hour mean reacts slowly by design. That is why rules use the instantaneous value for “right now” and the mean for “has this been going on”.

Look at the rows around the first start yourself (`df.iloc[98:106]` in pandas): static sits at 5.6 Pa, then 137.5 Pa for several rows in a row. The historian keeps the last value until

a reading changes enough (change-of-value logging), so repeated values are normal. Day 15 turns that into a quality check.

Try it

1. Why does AHU_1 have one more fan start than fan stop?
2. In both scripts change `min_periods=1` / the frame so that the first 11 rows of each AHU are `NULL`/`NaN` instead (pandas: delete `min_periods=1`; SQL: wrap the average in `CASE WHEN row_number() ... >= 12`). What do you get? Which is the "right" first-row behaviour for a rule that must see a full hour?
3. A step of more than 100 Pa in one row: how would you find those rows with the columns the script already makes?

Answers

1. The last fan run is still going when the data ends (2026-07-17 09:55 UTC), so it has a start and no stop. The first row has no previous row, so it can be neither a start nor a stop.
2. The first 11 rows become `NaN` (pandas) or `NULL` (SQL) for `static_1h_mean_pa`. For a rule that needs a full hour of history, that is right: a mean of the 3 rows that exist is not an hour mean. For a plot, a partial

mean is friendlier. Pick on purpose; the two tools default differently (SQL averages what exists, pandas returns `NaN`).

3. The scripts already make `static_prev_pa`. Add `df["static_step_pa"] = df["static_pa"] - df["static_prev_pa"]` and keep `df[df["static_step_pa"].abs() > 100]` in pandas; in SQL, `WHERE ABS(static_pa - static_prev_pa) > 100` on the windowed rows. That is 186 rows on AHU_1. Many are the fan starting (about +290 Pa) or stopping (about -290 Pa), and the jump lands one row after the status flips. Compare them with `fan_start` and `fan_stop`.

Why it matters

Almost every rule is a window function in disguise. “Fan started less than 15 minutes ago” (suppress alarms while the system settles), “static has been above setpoint for the last hour” (rolling mean), “the status changed” (LAG). Day 13 builds on `LAG` to count how long something stays the same, and Day 14’s confirmation step uses the result.

Day 13 — Streaks (gaps and islands): how long has this been true?

Goal: find runs of consecutive rows with the same value, using the three-step streak trick in SQL and its `cumsum` twin in pandas. Use it to find the longest continuous fan run and to count "true for N rows in a row", the confirmation step in every Open-FDD rule.

Read

A lot of FDD questions are about **duration**, not single readings: "the fan has been on for 30 minutes", "static has been low for an hour", "this sensor hasn't changed all night". A raw fault on one row is noise; a raw fault that holds for N rows is a finding. Rows in a table don't know their neighbours, so we need a way to say "these consecutive rows belong to one run". This is the classic *gaps and islands* problem.

The trick has three steps. Take the fan status as an example:

```
status        0 0 1 1 1 0 1 1
new_streak    1 0 1 0 0 1 1 0      1 where
the value differs from the previous row
(the first row is always 1)
streak_id     1 1 2 2 2 3 4 4      running
total of new_streak: every row of a run
shares one id
row in streak 1 2 1 2 3 1 1 2      counts
1, 2, 3 ... inside each run
```

Now a run is just a `GROUP BY streak_id`, and "true for N rows in a row" is `row in streak >= N`. Each step is cheap and the whole thing is one pass over the rows, O(n) after sorting.

Step	day13_sql.py (window functions)	day13_pandas.py (vectorized)
mark changes	`x IS NOT LAG(x) OVER (ORDER BY timestamp_utc)`	`x != x.shift()`
streak id	`SUM(new_streak) OVER (ORDER BY timestamp_utc)`	`(x != x.shift()).cumsum()`
row in streak	`ROW_NUMBER() OVER (PARTITION BY streak_id ORDER BY`	`x.groupby(streak_id).cumcount() + 1`

Step	day13_sql.py (window functions)	day13_pandas.py (vectorized)
	`timestamp_utc)`	
each run	`GROUP BY streak_id`	`groupby(streak_id).agg(...)`
longest run	`ORDER BY rows DESC, start_utc LIMIT 1`	`runs.loc[runs["rows"].idxmax()]`

Two details. On the first row `LAG` is `NULL`, and `x IS NOT NULL` is true (plain `x != NULL` would be `NULL`, not true): the first row starts a streak, as it should. In pandas `shift()` gives `NaN` there, and `x != NaN` is `True`. And a *tie* for the longest run needs a rule, or two runs of equal length make the answer unstable; both scripts take the earlier start. The scripts look at AHU_1; with both AHUs in one table every window would also need `PARTITION BY equipment_id` (pandas: `groupby("equipment_id")`), so a streak never runs from one AHU into the next.

A second route to "true for N rows in a row". Instead of numbering streaks, slide a window of N rows over the 0/1 raw flag and ask whether all N are 1: `SUM(raw) OVER (... ROWS BETWEEN 5 PRECEDING AND CURRENT ROW) = 6` in SQL, `raw.rolling(6).sum() == 6` in pandas

(Day 12's windows). Both scripts compute both routes and print their counts one under the other: they agree. The streak route needs no window size, and it gives you the streak id so you can count episodes and measure their length. The window route is shorter when you only need the flag.

Today's raw flag is "supply fan at 95% speed or more while running", confirmed after **6 rows** (30 minutes). Day 14 uses the same shape with a pressure rule.

Run

```
uv run python lessons/course.py run 13
```

That runs `day13_pandas.py`, then `day13_sql.py`. Both print:

```
AHU_1 fan runs (status = 1)
  runs: 123
  average run (hours): 12.8
  longest run (rows): 1313
  longest run (hours): 109.42
  longest run started: 2026-07-06
09:45:00+00:00
Full speed (>= 95 %) for 6 rows in a row
  raw rows: 1349
  confirmed rows (streak route): 1153
  confirmed rows (rolling route): 1153
  episodes: 31
```

The fan normally runs about 14 hours a day. The longest run is **109 hours**, from 6 July 09:45 to 10 July 23:05 UTC: the fan never stopped for over four days, so something (probably a schedule override) kept it on. AHU_2 has the same 109-hour run. A person reading a chart would find that; the streak trick finds it for every AHU in a building with one query.

The confirmed count is smaller than the raw count because the first 5 rows of each full-speed run are not yet "confirmed" (and one-row blips never are). AHU_1 spends far more time at full speed than AHU_2 (1,349 vs 222 raw rows), which is itself worth a question.

Try it

1. Change the confirmation from 6 rows to 12 (one hour) in both scripts. In SQL there are two places to change. What happens to the two routes if you only change `:confirm`?
2. Find the longest time each AHU was **off** (status 0) in a row.

Answers

1. With 12 rows, AHU_1 drops to 995 confirmed rows in 21 episodes. The streak route only needs `:confirm_rows` (or

`CONFIRM_ROWS`). The window route also needs its frame (`ROWS BETWEEN 11 PRECEDING`). Change only the parameter and the frame still holds 6 rows, so the window route finds none (SQL) while the streak route finds the new answer. The two columns disagree, which is how you'd spot the slip. pandas hides it, because `rolling(CONFIRM_ROWS)` reads the same constant.

2. Filter the streaks to `supply_fan_status = 0` instead of `1` and take the longest. It is 999 rows, 83.25 hours, on AHU_1 (and the same on AHU_2). The streak numbers already cover the runs of both kinds; only the filter changes.

Why it matters

Day 14 builds an entire fault rule from these pieces: a raw flag, the streak trick, `streak_len >= N`, and a sum of fault rows times five minutes. Day 15 reuses the trick to find flatlined sensors, where the "value" being compared is the sensor reading itself.

Day 14 — Write a fault rule (FC1, duct static too low) twice, and check it against Open-FDD

Goal: turn an engineering rule into a pandas script *and* a SQL script, see that they give the same answer, count fault hours, and compare with a production FDD engine's results.

Read

Fault detection and diagnostics (FDD) runs rules over trend data to find equipment that isn't doing what it should. Open-FDD's rules come from ASHRAE Guideline 36 and the NIST/PNNL APAR rules. **FC1** says:

> The supply fan is running (at or near full speed) but duct static pressure stays below its setpoint.

The fan is working as hard as it can and still can't make pressure. Likely causes: a broken belt, a failing VFD, a blocked filter or coil, a leaky duct, or a setpoint too high for this system.

Every Open-FDD rule (`sql/fdd/postgres/rules/*.sql`, and its pandas twin in `open_fdd/rules`) has the same shape:

1. **Gate**: only judge rows where the equipment is on (`supply_fan_status = 1`). Off-hours aren't faults.
2. **Raw fault**: `CASE WHEN <condition> THEN 1 ELSE 0 END`, with a tolerance so noise doesn't trip it.
3. **Confirm**: count a row only once the raw fault has held for N consecutive rows (Day 13's streaks).
4. **Aggregate**: fault rows × 5 minutes = fault hours.

The math is **SI**, like Open-FDD (Day 6): the CSV's `da_p_inwc` is in inches of water, so both scripts convert to pascals first. The tolerance is **25 Pa** (about 0.1 inH2O) and a fault is confirmed after **2 rows** (10 minutes).

The inputs are **hard-coded column names**: `supply_fan_status`, `da_p_inwc`, `da_p_setpoint_inwc`. The scripts run on AHU_1; AHU_2's export happens to use the same names, so pointing them at AHU_2 is a one-word change. That's fine for one building. Week 5 shows where it breaks, and how a data model replaces those names.

Open-FDD keeps every rule twice on purpose. SQL is what runs next to the data. pandas is the **oracle**: a second, independent implementation, easy to step through in a notebook. Its parity tests run both on the same data and fail if a single row differs. Today you write both halves and compare them.

How each step maps across:

Step	`day14_sql.py` (per row, window functions)	`day14_pandas.py` (per column, vectorized)
1. read AHU_1	`FROM ahu_1`	`pd.read_csv("...AHU_1/history_wide.csv", usecols=columns)`
2. SI	`da_p_inwc * :pa_per_inh2o AS static_pa`	`df["da_p_inwc"] * PA_PER_INH2O`
3. gate + raw fault	`supply_fan_status = 1 AND static_pa < setpoint_pa - :tolerance_pa`	`fan_on & too_low`
4a. “value changed”	`raw_fault IS NOT LAG(raw_fault) OVER (ORDER`	`df["raw_fault"] != df["raw_fault"].shift()`

Step	`day14_sql.py` (per row, window functions)	`day14_pandas.py` (per column, vectorized)
	`BY timestamp_utc )`	
4b. streak id	`SUM(new_streak) OVER (ORDER BY timestamp_utc )`	`df["new_streak"].cumsum()`
4c. row within streak	`ROW_NUMBER() OVER (PARTITION BY streak_id ORDER BY timestamp_utc )`	`df.groupby("streak_id").cumcount() + 1`
5. hours	`SUM(fault) * 5 / 60.0`	`df["fault"].sum() * 5 / 60`

Both are O(n) once the rows are in time order. With several AHUs in one table, every window would add `PARTITION BY equipment_id` (pandas: `groupby("equipment_id")`), so a streak never runs from one AHU into the next. Day 20 and Day 25 do exactly that.

Run

```
uv run python lessons/course.py run 14
```

That runs `day14_pandas.py`, then `day14_sql.py`. Read them side by side: each numbered comment in one has the same number in the other. Both print:

```
AHU_1, FC1 (duct static too low)
  raw fault hours:       120.0
  confirmed fault hours: 114.25
  confirmed episodes:    53
Open-FDD, FC1 on AHU_1
  episodes: 80
  hours:    484.1
Open-FDD episodes that contain one of our
faults: 73 of 80
```

AHU_1 has about 114 confirmed fault hours in 53 episodes. Point the scripts at AHU_2 (`AHU_2` in the CSV paths, `ahu_2` and `bld_50_AHU_2` in the SQL) and you get 1.5 hours in 5 episodes: essentially healthy for this rule (its problem is the opposite, static *above* setpoint, as Day 8 showed).

AHU_1 has 1,371 confirmed rows = 114.25 h exactly, a tie at one decimal. SQLite's `round(114.25, 1)` rounds halves away from zero (114.3), numpy and Python round halves to even (114.2). Round only for display, never before comparing two results.

Compare with Open-FDD. Its results for this data ship next to the CSVs in `data/BUILDING_50/*/fdd_events.csv` (the SQL

side has them as the `fdd_events` table). It found **80 FC1 episodes, 484 hours** on AHU_1, against our 53 and 114. Different numbers don't mean either is wrong. Check whether the two engines are finding **the same events**: how many of Open-FDD's episodes contain at least one of our confirmed fault rows? (Step 6.)

- SQL uses a **correlated `EXISTS`**: for each of the E events, ask "is there a confirmed fault row between this event's start and end?". Without an index each probe scans the `fc1` table; add one (`CREATE INDEX temp.fc1_time ON fc1 (timestamp_utc)`) and each probe is a B-tree range seek: O(E log N).
- pandas has no `EXISTS`, so the script loops over the 80 events and tests the fault times against each one (`inside.any()`): E passes over our F fault rows, O(E · F). That is instant for 80 events. With thousands you would sort the fault times once and binary-search them (`np.searchsorted`), O(E log F).

Both print **73 of 80**. Same problems, measured differently. Open-FDD's episode clock runs from first to last fault sample, including stretches in between, and its event timestamps aren't on this 5-minute grid. It ran on its own copy of the data with its own

tuning. Totals depend on those choices; overlap tells you whether the rules agree on **what** is wrong.

Real Open-FDD rules (`cmd1_fan_mismatch.sql` is a short one to read) add one more step: **forward sample-and-hold with a gap limit** (`MAX_GAP_HOURS`). Building 50 has no gaps (every step is exactly 5 minutes), so counting rows works here. Day 15 checks that, and Day 18 builds the gap-aware version.

Try it

1. Change the tolerance from 25 Pa to 50 Pa (~0.2 inH2O) in both scripts (`TOLERANCE_PA` in pandas, `"tolerance_pa"` in SQL). How much do AHU_1's confirmed hours drop? Do the two scripts still agree?
2. Write the opposite rule, **duct static too high** (`duct_static > setpoint + 25 Pa` while running), in both scripts, and run it on AHU_2.

Answers

1. From 114.25 to 101.25 hours. Most of AHU_1's shortfall is more than 50 Pa: a real problem, not a marginal one.
2. Point the scripts at AHU_2 and flip one comparison in each: `df["static_pa"] > df["setpoint_pa"] + TOLERANCE_PA` and

`static_pa > setpoint_pa + :tolerance_pa`. AHU_2 shows about 1,428 confirmed hours, nearly all of its 1,500 run hours. The fan pushes far more pressure than its setpoint asks for, all the time: wasted fan energy, and possibly a reset sequence that isn't working. Open-FDD's version of this rule is `ahu_ducthi.sql`.

Why it matters

Notice what the rule needed from outside the data: which column is "duct static" and which is "fan status" for each AHU. Here those names are typed into the code. Change buildings, or meet an AHU whose export names a column differently, and the rule breaks even though the physics hasn't changed. Week 5 fixes that with a semantic model: the rule stays the same, and the model tells it where its inputs are. And because you have the rule twice, a change to either script that alters a single row shows up as a mismatch (`tests/test_lesson_scripts.py` checks every day).

Day 15 — Data quality before you trust a rule, and the Unit 1 challenge

Goal: check the data before judging the equipment: dropouts, a setpoint that reads zero, out-of-range values, flatlined sensors, the time step, and coverage. Then finish Unit 1 by writing a discharge air temperature rule in both pandas and SQL.

Read

A fault rule can only be as good as its inputs. A rule that fires on a sensor dropout reports a broken *logger*, not a broken *air handler*, and the person who gets the work order can't tell the difference. So before a rule, check the data. Today's checks run on AHU_1, in both scripts, with the same answers. They are `day15_pandas.py` and `day15_sql.py`. (Point them at AHU_2 to compare; its numbers are quoted below too.)

Check	What it looks for	pandas	SQL
3. dropout	a temperature that	`(df["discharge_air_temp_f"]`	`discharge_air_temp`

Check	What it looks for	pandas	SQL
	reads exactly 0 °F	`== 0) | ...`	`_f = 0 OR ...`
4. missing setpoint	setpoint reads 0 while the fan runs	`fan_on & (df["dat_reset_f"] == 0)`	`supply_fan_status = 1 AND dat_reset_f = 0`
5. out of range	outside what a sensor can report	`(x < -40) | (x > 60)`	`x NOT BETWEEN :temp_min_c AND :temp_max_c`
6. flatline	an identical value for 1 h or more	Day 13's streaks: `(x != x.shift()).cumsum()`	Day 13's window steps
7. time step	every row 300 s after the last	`pd.to_datetime(...).diff()`	`unixepoch(ts) - LAG(unixepoch(ts)) OVER w`

All temperatures are converted to °C (`(°F − 32) × 5/9`) and static to Pa before the checks, like Day 14. The dropout test is the one place where we look at the raw °F value on purpose.

Dropouts, and the comparison trap

Seven rows on each AHU have the mixed, return and discharge temperatures all reading **exactly 0 °F**. A logger writes 0 when it gets no reading; 0 °F is a perfectly valid number, so nothing flags it. After conversion that 0 °F is **-17.78 °C**, and that is where the trap is.

```
df["discharge_air_temp_f"] == 0  # 7 rows:
        the raw value is exactly 0.0
df["dat_c"] == -17.78  # 0 rows: the
        converted value is
        -17.7777777...
(df["dat_c"] - ZERO_F_IN_C).abs() < 0.01
        # 7 rows: compare with a
        tolerance
```

-17.78 is a rounded, printed number. The value in memory is -17.777777... and floating-point == against any rounded constant never matches. Two safe ways: test the **raw value** before converting (exact zeros are stored exactly), or compare converted values with a **tolerance**. Never == a float you computed. The scripts print both the exact-match count (0) and the tolerance count (7), one under the other.

The seven rows form three episodes: one row on 31 March, five on 22 April (16:10 to 16:30 UTC), one on 15 June. Notice that they are not caught by the **range check**: our sensor range is -40 to 60 °C, and -17.78 °C is inside it. A

range check catches impossible values (a duct at 500 °C); a dropout is a *plausible wrong* value, which is harder. Out-of-range is 0 rows here. That is a result too.

A setpoint of zero, and flatlines

`dat_reset_f` is the discharge air temperature **setpoint**. While the fan runs it should sit between 50 and 71 °F. It reads **0 °F** for 1,233 running rows on AHU_1 and 984 on AHU_2, mostly in June. Seven of those are the dropout rows. The rest are the control system publishing no setpoint. A rule comparing the discharge temperature with a setpoint of -17.78 °C would call the whole stretch a fault.

Flatlines. A working temperature sensor wiggles in its last digits. The same value for an hour or more while the fan runs is suspicious. This is Day 13's streak trick on the sensor reading: streaks of an identical `discharge_air_temp_f`, kept if the streak is at least 12 rows (1 hour) and the fan ran for all of it. That finds 66 episodes (1,333 rows) on AHU_1 and 48 (1,064 rows) on AHU_2, the longest 136 and 135 rows (11 hours). Remember Day 12: the historian logs change-of-value, so a stable reading is *normally* repeated. Treat flatlines as "look here", not as proof, and raise the threshold if it's too chatty (see Try it).

The time step, and coverage

Every row should be exactly 300 s after the one before. The check takes the difference to the previous timestamp and counts rows where it isn't 300: **0 gaps, max step 300 s** on both AHUs. That is why counting rows works as counting time in Days 12 to 14 (`ROWS BETWEEN 11 PRECEDING` really is one hour). Real data usually has gaps. Day 18 builds the gap-aware version.

Coverage per month (Try it 3) compares the rows present with the rows expected from the first to the last timestamp in the month: 100% for every month here. March and July are partial months (the export starts on 16 March and ends on 17 July), so compare coverage against the *span*, not against a calendar month.

Run

```
uv run python lessons/course.py run 15
```

Both scripts print the same numbers (pandas shows `300.0` where SQL shows `300`):

```
AHU_1 data-quality checks
  dropout rows (a temperature reads 0 F):
7
  setpoint reads 0 while the fan runs:
1233
  discharge temp out of range:
0
  flatline rows (1 h+ the same value):
1333
  flatline episodes:
66
  time steps that are not 300 s:
0
  longest time step (s):
300.0
  dat_c == -17.78 matches:
0
  dat_c within 0.01 of -17.78 matches:
7
```

followed by the challenge answer below.

Try it

1. Raise the flatline threshold from 12 rows to 36 (3 hours), in both scripts (`FLAT_ROWS` in pandas, `flat_rows` in SQL). How many episodes are left?
2. The `dropout` flag looks at three temperatures. Why is it safer to flag the *row* than to remove the one dropout sensor?
3. Coverage: for each month, count the rows present and the rows expected from the

month's first to its last timestamp (`span_s / 300 + 1`, Day 3). What percentage is present?

Answers

1. Five episodes remain on AHU_1 (419 rows; AHU_2 also has five, 416 rows). Most hour-long flatlines are probably change-of-value repeats. The few stretches of 3 hours or more are the ones worth asking about.
2. In a dropout all three sensors fail at the same moment, which points at the controller or the network rather than one sensor. A rule that mixes several temperatures would otherwise compute with a mix of real and fake values. Flag the whole row and let each rule skip it.
3. 100% for every month. In pandas: `df.groupby("month")["time"].agg(["size", "min", "max"])` with `time = pd.to_datetime(df["timestamp_utc"])`, then `(max - min).dt.total_seconds() / 300 + 1`. In SQL: `GROUP BY strftime('%Y-%m', timestamp_utc)` with `COUNT(*)` and `(unixepoch(MAX(...)) - unixepoch(MIN(...))) / 300 + 1`.

Unit 1 challenge: a discharge air temperature rule, in SI, with a quality gate

Write the rule in both scripts (it's already in both, as steps 8 and 9, so the parity test covers it; try writing it yourself first):

> While the supply fan runs, the discharge air temperature is **more than 2.78 °C (5 °F) away from its setpoint** (`discharge_air_temp_f` vs `dat_reset_f`), and this holds for **30 minutes (6 rows)**.

Report **fault hours per month** (AHU_1; then AHU_2). Do it twice: once trusting every row, once skipping rows where the data quality checks fail.

Answer

Same shape as Day 14: gate, raw fault, confirm, aggregate.

```
error_c = (df["dat_c"] -
          df["dat_setpoint_c"]).abs()  #
          both converted: (F - 32) * 5 / 9
raw_ungated = fan_on & (error_c >
          TOLERANCE_C)
raw_gated = raw_ungated & ~df["dropout"] &
          ~df["setpoint_zero"]
# then Day 13's streak trick: fault = raw
          & (row_in_streak >= 6); hours =
          rows * 5 / 60
```

```
supply_fan_status = 1 AND ABS(dat_c -
          dat_setpoint_c) > :tolerance_c
          -- ungated
supply_fan_status = 1 AND ABS(dat_c -
          dat_setpoint_c) > :tolerance_c
  AND NOT dropout AND NOT setpoint_zero
          -- gated
-- LAG / SUM OVER / ROW_NUMBER as on Day
          13, then SUM(fault) * 5 / 60.0
          per month
```

The absolute value matters: the discharge temperature can be too high or too low. A *difference* of temperatures converts with only the 5/9 factor (the 32 cancels), so 5 °F is 5 × 5/9 = 2.78 °C. The scripts convert each temperature first and then subtract, which gives the same answer and keeps every column in °C.

Fault hours on AHU_1 (ungated / gated), as the scripts print them:

```
  month  ungated_fault_h  gated_fault_h
2026-03            22.25          22.25
2026-04            46.00          46.00
2026-05           113.50         113.50
2026-06           213.75         123.75
2026-07            21.00          11.33
  total, trusting every row (h): 416.5
  total, quality gate on (h):    316.83
```

AHU_2, with the scripts pointed at it: 298.00 h ungated, 218.25 h gated (June 174.75 -> 104.67).

Both AHUs miss their discharge setpoint for a good part of the time, most of all in June. The quality gate matters: the missing setpoint adds about 100 hours on AHU_1 and 80 on AHU_2, roughly a quarter of the ungated totals, almost all of it in June and July, because a setpoint of -17.78 °C is “wrong” by 30 °C whenever the fan is on. Those are not faults in the air handler. March and April are unchanged: the 7 dropout rows are too few and too short to reach the 6-row confirmation. Fixing the setpoint publisher is its own work order.

Why it matters

The quality gate changed the answer by about a quarter. That’s the point of the day: **read the data before you read the rule’s**

output. Production FDD does the same checks on every run. The gap-aware sample-and-hold (`MAX_GAP_HOURS`) comes on Day 18.

What production does differently (Timescale). Everything this unit used (CSV, SQLite) is one machine and a few hundred thousand rows. Open-FDD's production stack keeps history in **PostgreSQL with the TimescaleDB extension**: the same SQL (window functions, `CASE`, `GROUP BY` all carry over), but the table is split into time chunks, old chunks compress, and queries for one series and one time range touch only their chunks. That is Day 11's index idea, applied automatically. Hard-coded column names are the other thing that changes: Week 5 replaces `discharge_air_temp_f` with a lookup in a building model, so one rule runs over many buildings and exports.

Reading — Week 3

The lessons first, then these for exact behavior.

- SQLite window functions
- pandas: windowing operations
- SQLite query planner
- SQLite EXPLAIN QUERY PLAN
- Open-FDD SQL cookbook
- TimescaleDB hypertables
- TimescaleDB continuous aggregates